

Mestre do PC V7 - Ultimate

Visao Geral

Interface monolitica com tema dark/cyber, 14 categorias de manutencao, 142 comandos PowerShell, integracao com Ollama e launcher HTTP na porta 7777.

Funcoes JavaScript

- toggleCat(header): expande/colapsa secao de categoria.
- cancelParam(): fecha overlay de parametro e limpa estado pendente.
- confirmParam(): le valor do input, substitui token no comando e executa.
- showToast(msg, color): exibe notificacao temporaria.
- checkServer(): consulta /ping do launcher e atualiza badge de status.
- sleep(ms): utilidade de espera.
- dispatchCommand(code, options): envia POST /run e faz polling em /run-status.
- copyById(uid): copia conteudo de elemento para clipboard.
- runCmd(uid): obtem comando do card, verifica padroes de parametro, executa ou copia.
- executeCmd(code): exibe painel de output, dispara comando e trata resultado.
- openPS(): abre terminal PowerShell admin via /open-terminal.
- chooseOllamaModel(models): seleciona modelo preferencial da lista.
- openAI(): abre overlay de chat IA.
- closeAI(): fecha overlay de chat IA.
- checkOllama(): lista modelos disponiveis em /api/tags.
- sendAI(): envia mensagem para Ollama em streaming.
- addIAMessage(text, role): adiciona mensagem no chat.
- renderIAContent(el, text): converte markdown e blocos de codigo em HTML executavel.
- runIACmd(btn): executa comando PowerShell sugerido pela IA.
- scrollIAToBottom(): rola chat ate final.
- showOutput(text): exibe texto no painel de output.

Comandos PowerShell por Categoria

1. Limpeza Geral (13 comandos)

Limpeza Rápida Completa

```
Remove-Item "$env:TEMP\*" -Recurse -Force -EA 0; Remove-Item "C:\Windows\Temp\*" -Recurse -Force -EA 0; Clear-RecycleBin -Force -EA 0; Write-Host "✅ Limpeza concluida!" -ForegroundColor Green
```

Limpar TEMP Usuário

```
Remove-Item -Path "$env:TEMP\*" -Recurse -Force -ErrorAction SilentlyContinue
```

Limpar TEMP do Windows

```
Remove-Item -Path "C:\Windows\Temp\*" -Recurse -Force -ErrorAction SilentlyContinue
```

Esvaziar Lixeira

```
Clear-RecycleBin -Force -ErrorAction SilentlyContinue; Write-Host "✅ Lixeira esvaziada!" -ForegroundColor Green
```

Limpar Cache do Windows Update

```
Stop-Service wuauclt -Force; Remove-Item "C:\Windows\SoftwareDistribution\Download\*" -Recurse -Force -EA 0; Start-Service wuauclt; Write-Host "✅ Cache WU limpo!" -ForegroundColor Green
```

Limpar Prefetch

```
Remove-Item "C:\Windows\Prefetch\*" -Force -EA 0; Write-Host "✅ Prefetch limpo!" -ForegroundColor Green
```

Limpar Minidump (crash logs)

```
Remove-Item "C:\Windows\Minidump\*" -Force -EA 0; Remove-Item "$env:LOCALAPPDATA\CrashDumps\*" -Recurse -Force -EA 0; Write-Host "✅ Dumps limpos!" -ForegroundColor Green
```

Limpar Cache DNS

```
ipconfig /flushdns; Write-Host "✅ Cache DNS limpo!" -ForegroundColor Green
```

Limpeza Profunda (tudo)

```
Remove-Item "$env:TEMP\*" -Recurse -Force -EA 0; Remove-Item "C:\Windows\Temp\*" -Recurse -Force -EA 0; Remove-Item "C:\Windows\Prefetch\*" -Force -EA 0; Clear-RecycleBin -Force -EA 0; ipconfig /flushdns; Write-Host "✅ Limpeza profunda concluida!" -ForegroundColor Green
```

Limpar Cache do Chrome

```
Remove-Item "$env:LOCALAPPDATA\Google\Chrome\User Data\Default\Cache\*" -Recurse -Force -EA 0; Remove-Item "$env:LOCALAPPDATA\Google\Chrome\User Data\Default\Code Cache\*" -Recurse -Force -EA 0; Write-Host "✅ Cache Chrome limpo!" -ForegroundColor Green
```

Limpar Cache do Teams

```
Remove-Item "$env:APPDATA\Microsoft\Teams\Cache\*" -Recurse -Force -EA 0; Remove-Item "$env:APPDATA\Microsoft\Teams\blob_storage\*" -Recurse -Force -EA 0; Remove-Item "$env:APPDATA\Microsoft\Teams\databases\*" -Recurse -Force -EA 0; Write-Host "✅ Cache Teams limpo!" -ForegroundColor Green
```

Limpar Arquivos Antigos do Downloads (>30 dias)

```
Get-ChildItem "$env:USERPROFILE\Downloads" -File | Where-Object { $_.LastWriteTime -lt (Get-Date).AddDays(-30) } | Remove-Item -Force -EA 0; Write-Host "✅ Downloads antigos removidos!" -ForegroundColor Green
```

Limpar Cache de Fontes

```
Stop-Service FontCache -Force -EA 0; Remove-Item "$env:WINDIR\ServiceProfiles\LocalService\AppData\Local\FontCache\*" -Recurse -Force -EA 0; Start-Service FontCache; Write-Host "✅ Cache de fontes limpo!" -ForegroundColor Green
```

2. Limpeza Avançada (12 comandos)

Cleanmgr Modo Silencioso

```
cleanmgr /sagerun:1; Write-Host "✅ Limpeza de disco iniciada!" -ForegroundColor Green
```

Configurar Cleanmgr Automático

```
cleanmgr /sageset:1
```

Limpar Arquivos de Log do Windows

```
Remove-Item "C:\Windows\Logs\*" -Recurse -Force -EA 0; Write-Host "✅ Logs limpos!" -ForegroundColor Green
```

Limpar EventViewer Logs

```
Get-EventLog -List | ForEach-Object { Clear-EventLog -LogName $_.Log -EA 0 }; Write-Host "✅ Logs de eventos limpos!" -ForegroundColor Green
```

Limpar Cache do Microsoft Store

```
wsreset.exe; Write-Host "✅ Cache do Store limpo!" -ForegroundColor Green
```

Limpar Cache do Navegador Edge

```
Remove-Item "$env:LOCALAPPDATA\Microsoft\Edge\User Data\Default\Cache\*" -Recurse -Force -EA 0; Write-Host "✅ Cache Edge limpo!" -ForegroundColor Green
```

Limpar Thumbnail Cache

```
ie4uinit.exe -show; Remove-Item "$env:LOCALAPPDATA\Microsoft\Windows\Explorer\thumbcache_*" -Force -EA 0; Write-Host "✅ Thumbnail cache limpo!" -ForegroundColor Green
```

Compactar WinSxS (DISM)

```
DISM /Online /Cleanup-Image /StartComponentCleanup /ResetBase
```

Limpar Arquivos Desnecessários do Sistema

```
DISM /Online /Cleanup-Image /StartComponentCleanup
```

Limpar Cache do Spotify

```
Remove-Item "$env:APPDATA\Spotify\Storage\*" -Recurse -Force -EA 0; Write-Host "✅ Cache Spotify limpo!" -ForegroundColor Green
```

Limpar Cache do Discord

```
Remove-Item "$env:APPDATA\discord\Cache\*" -Recurse -Force -EA 0; Remove-Item "$env:APPDATA\discord\Code Cache\*" -Recurse -Force -EA 0; Write-Host "✅ Cache Discord limpo!" -ForegroundColor Green
```

Remover Arquivos .tmp do Sistema

```
Get-ChildItem C:\ -Recurse -Filter *.tmp -EA 0 | Where-Object { !$_.PSIsContainer } | Remove-Item -Force -EA 0; Write-Host "✅ Arquivos .tmp removidos!" -ForegroundColor Green
```

3. Memória / RAM (7 comandos)

Liberar Memória RAM Imediatamente

```
[System.GC]::Collect(); [System.GC]::WaitForPendingFinalizers(); Write-Host "✅ RAM liberada!" -ForegroundColor Green
```

Ver Uso Atual de RAM

```
$ram = Get-WmiObject Win32_OperatingSystem; $livre = [math]::Round($ram.FreePhysicalMemory/1MB,2); $total = [math]::Round($ram.TotalVisibleMemorySize/1MB,2); Write-Host "RAM Livre: $livre GB de $total GB" -ForegroundColor Cyan
```

Listar Processos por Uso de RAM

```
Get-Process | Sort-Object WorkingSet64 -Descending | Select-Object -First 15 Name,@{N="RAM(MB)";E=[math]::Round($_.WorkingSet64/1MB,2)} | Format-Table -AutoSize
```

Desativar SuperFetch (SysMain)

```
Set-Service -Name "SysMain" -StartupType Disabled; Stop-Service -Name "SysMain" -Force; Write-Host "✅ SysMain desativado!" -ForegroundColor Green
```

Reativar SuperFetch (SysMain)

```
Set-Service -Name "SysMain" -StartupType Automatic; Start-Service -Name "SysMain"; Write-Host "✅ SysMain reativado!" -ForegroundColor Green
```

Verificar Modo de Memória Virtual

```
$cs = Get-ComputerInfo; Write-Host "Paginação: $($cs.OsMaxProcessMemorySize) KB" -ForegroundColor Cyan
```

Limpar Standby Memory (RAMMap)

```
Write-Host "Baixe RAMMap da Sysinternals para limpar Standby Memory manualmente." -ForegroundColor Yellow
```

4. Processos (9 comandos)

Listar Processos Ativos

```
Get-Process | Sort-Object CPU -Descending | Select-Object -First 20 Name, Id, CPU, @{N="RAM(MB)";E={ [math]::Round($_.WorkingSet64/1MB,1) }} | Format-Table -AutoSize
```

Encerrar Processo por Nome

```
Stop-Process -Name "NOME_DO_PROCESSO" -Force -EA 0; Write-Host "☑ Processo encerrado!" -ForegroundColor Green
```

Encerrar Todos Chrome

```
Stop-Process -Name "chrome" -Force -EA 0; Write-Host "☑ Chrome encerrado!" -ForegroundColor Green
```

Encerrar Todos Edge

```
Stop-Process -Name "msedge" -Force -EA 0; Write-Host "☑ Edge encerrado!" -ForegroundColor Green
```

Listar Serviços em Execução

```
Get-Service | Where-Object Status -eq "Running" | Sort-Object DisplayName | Format-Table -AutoSize
```

Listar Startups do Sistema

```
Get-CimInstance Win32_StartupCommand | Select-Object Name,Command,User | Format-Table -AutoSize
```

Desativar Serviço por Nome

```
Set-Service -Name "NOME_SERVICO" -StartupType Disabled; Stop-Service "NOME_SERVICO" -Force
```

Reiniciar Explorer

```
Stop-Process -Name explorer -Force; Start-Process explorer; Write-Host "☑ Explorer reiniciado!" -ForegroundColor Green
```

Ver tarefas agendadas

```
Get-ScheduledTask | Where-Object State -eq "Ready" | Select-Object TaskName,TaskPath | Format-Table -AutoSize
```

5. Disco (9 comandos)

Verificar Espaço em Disco

```
Get-PSDrive -PSProvider FileSystem | Select-Object Name, @{N="Usado(GB)";E={ [math]::Round($_.Used/1GB,2) } }, @{N="Livre(GB)";E={ [math]::Round($_.Free/1GB,2) } }, @{N="Total(GB)";E={ [math]::Round((($_.Used+$_.Free)/1GB),2) } } | Format-Table -AutoSize
```

CHKDSK Verificação Rápida

```
chkdsk C: /f /r; Write-Host "☑ CHKDSK agendado para o próximo boot!" -ForegroundColor Green
```

Otimizar / Desfragmentar Disco

```
Optimize-Volume -DriveLetter C -Defrag -Verbose
```

Trim SSD

```
Optimize-Volume -DriveLetter C -ReTrim -Verbose; Write-Host "☑ TRIM executado!" -ForegroundColor Green
```

Verificar Saúde do Disco (SMART)

```
Get-WmiObject -Namespace root/wmi -Class MSStorageDriver_FailurePredictStatus | Select-Object PredictFailure,Reason | Format-Table -AutoSize
```

Encontrar Pastas Gigantes (C:)

```
Get-ChildItem C: -Directory | ForEach-Object { $s=(Get-ChildItem $_.FullName -Recurse -EA 0 | Measure-Object -Property Length -Sum).Sum; [PSCustomObject]@{Pasta=$_.Name;GB=[math]::Round($s/1GB,2) } | Sort-Object GB -Desc | Select-Object -First 15 | Format-Table
```

Relatório de Disco (WinSAT)

```
winsat disk -drive C
```

Desativar Hibernação (libera espaço)

```
powercfg -h off; Write-Host "☑ Hibernação desativada, hiberfil.sys removido!" -ForegroundColor Green
```

Ativar Hibernação

```
powercfg -h on; Write-Host "☑ Hibernação ativada!" -ForegroundColor Green
```

6. Rede Básica (9 comandos)

Diagnóstico de Rede Completo

```
ipconfig /all
```

Testar Conectividade com a Internet

```
Test-Connection -ComputerName 8.8.8.8 -Count 4 | Format-Table -AutoSize
```

Velocidade de Download (Teste rápido)

```
Invoke-WebRequest -Uri "http://speedtest.tele2.net/1MB.zip" -OutFile "$env:TEMP\test.zip" -UseBasicParsing; Write-Host "✅ Download concluído!" -ForegroundColor Green
```

Flush e Renovar IP

```
ipconfig /flushdns; ipconfig /release; ipconfig /renew; Write-Host "✅ IP renovado!" -ForegroundColor Green
```

Resetar Pilha TCP/IP

```
netsh int ip reset; netsh winsock reset; Write-Host "✅ TCP/IP resetado! Reinicie o PC." -ForegroundColor Green
```

Ver Conexões Ativas

```
netstat -ano | Select-String "ESTABLISHED"
```

Ver Adaptadores de Rede

```
Get-NetAdapter | Select-Object Name,Status,LinkSpeed,MacAddress | Format-Table -AutoSize
```

Testar DNS

```
Resolve-DnsName google.com; Resolve-DnsName cloudflare.com
```

Ping para múltiplos servidores

```
"8.8.8.8","1.1.1.1","208.67.222.222" | ForEach-Object { $r=Test-Connection $_ -Count 1 -EA 0; Write-Host "$_ -> $($r.ResponseTime)ms" }
```

7. Reparo do Sistema (11 comandos)

SFC Scan (Reparo de Arquivos)

```
sfc /scannow
```

DISM Restaurar Saúde

```
DISM /Online /Cleanup-Image /RestoreHealth
```

DISM Verificar Saúde

```
DISM /Online /Cleanup-Image /CheckHealth
```

DISM Scan Saúde

```
DISM /Online /Cleanup-Image /ScanHealth
```

Reparar Windows Update

```
net stop wuauerv; net stop cryptSvc; net stop bits; net stop msiserver; Remove-Item "C:\Windows\SoftwareDistribution" -Recurse -Force -EA 0; net start wuauerv; net start cryptSvc; net start bits; net start msiserver; Write-Host "✅ WU reparado!" -ForegroundColor Green
```

Resetar Políticas de Grupo

```
secedit /configure /cfg %windir%\inf\defltbase.inf /db defltbase.sdb /verbose
```

Reparar .NET Framework

```
DISM /Online /Enable-Feature /FeatureName:NetFx3 /All
```

Verificar Erros de Boot

```
bootrec /scanos; bootrec /fixmbr; bootrec /fixboot; bootrec /rebuildbcd
```

Reparar Registro do Windows

```
$ErrorActionPreference="SilentlyContinue"; $reg="HKLM:\SOFTWARE\Microsoft\Windows NT\CurrentVersion"; Write-Host "Versão: $($($reg).ProductName)" -ForegroundColor Cyan
```

Atualizar PowerShell para 7+

```
winget install Microsoft.PowerShell --accept-source-agreements --accept-package-agreements; Write-Host "✅ Atualização do PowerShell iniciada!" -ForegroundColor Green
```

Atualizar Terminais (CMD, PowerShell, Windows Terminal)

```
winget upgrade Microsoft.WindowsTerminal --accept-source-agreements --accept-package-agreements; winget upgrade Microsoft.PowerShell --accept-source-agreements --accept-package-agreements; Write-Host "✅ Atualização concluída." -ForegroundColor Green
```

🛡️ Saúde do PC (19 comandos)

Relatório rápido do PC

```
$os=Get-WmiObject Win32_OperatingSystem; $cpu=Get-WmiObject Win32_Processor | Select-Object -First 1; $disk=Get-PSDrive C; $ramLivre=[math]::Round($os.FreePhysicalMemory/1MB,2); $ramTotal=[math]::Round($os.TotalVisibleMemorySize/1MB,2); $diskLivre=[math]::Round($disk.Free/1GB,2); $diskTotal=[math]::Round(($disk.Used+$disk.Free)/1GB,2); $boot=[Management.ManagementDateTimeConverter]::ToDateTime($os.LastBootUpTime); $uptime=New-TimeSpan -Start $boot -End (Get-Date); Write-Host "===== RELATORIO RAPIDO DO PC =====" -ForegroundColor Cyan; Write-Host "Computador: $env:COMPUTERNAME"; Write-Host "Windows: $($os.Caption) Build $($os.BuildNumber)"; Write-Host "CPU: $($cpu.Name)"; Write-Host "RAM: $ramLivre GB livre de $ramTotal GB"; Write-Host "Disco C: $diskLivre GB livre de $diskTotal GB"; Write-Host "Uptime: $($uptime.Days)d $($uptime.Hours)h"; Write-Host ""; Write-Host "Top 10 processos por RAM:" -ForegroundColor Yellow; Get-Process | Sort-Object WorkingSet64 -Descending | Select-Object -First 10 Name,@(N="RAM(MB)";E={[math]::Round($_.WorkingSet64/1MB,2)})} | Format-Table -AutoSize; Write-Host ""; if (Test-Connection 8.8.8.8 -Count 1 -Quiet){Write-Host "Internet: OK" -ForegroundColor Green}else{Write-Host "Internet: SEM CONEXAO" -ForegroundColor Red}
```

Exportar diagnóstico para logs

```
$project="C:\Users\Jeanc\MestreDoPC_V7"; $logDir=Join-Path $project "logs"; New-Item -ItemType Directory -Force -Path $logDir | Out-Null; $file=Join-Path $logDir ("diagnostico-"+(Get-Date -Format "yyyyMMdd-HHmms")+".txt"); $os=Get-WmiObject Win32_OperatingSystem; $disk=Get-PSDrive C; $top=Get-Process | Sort-Object WorkingSet64 -Descending | Select-Object -First 10 Name,@(N="RAM(MB)";E={ [math]::Round($_.WorkingSet64/1MB,2)})} | Out-String; @("===== DIAGNOSTICO MESTRE DO PC =====", "Data: $(Get-Date)", "Computador: $env:COMPUTERNAME", "Windows: $($os.Caption) Build $($os.BuildNumber)", "RAM livre GB: $($([math]::Round($os.FreePhysicalMemory/1MB,2)))", "Disco C livre GB: $($([math]::Round($disk.Free/1GB,2)))", "", $top) | Set-Content -Path $file -Encoding UTF8; Write-Host "Diagnostico salvo em: $file" -ForegroundColor Green
```

Listar modelos Ollama

```
try { $data=(Invoke-RestMethod -Uri "http://localhost:11434/api/tags" -TimeoutSec 5); Write-Host "Modelos Ollama instalados:" -ForegroundColor Cyan; if(-not $data.models){ Write-Host "Nenhum modelo retornado pelo Ollama." -ForegroundColor Yellow } else { $data.models | ForEach-Object { $cloud=if($_.name -match "cloud"){ " [cloud]" } else { "" }; Write-Host ("~ " + $_.name + $cloud) } } } catch { Write-Host "Ollama offline ou inacessivel em localhost:11434" -ForegroundColor Yellow; Write-Host $_.Exception.Message }
```

Testar Ollama (qwen3-coder-next:cloud)

```
$model="qwen3-coder-next:cloud"; try { $body=@{model=$model;messages=@(@{role="user";content="Responda somente OK"});stream=$false} | ConvertTo-Json -Depth 6; $res=Invoke-RestMethod -Uri "http://localhost:11434/api/chat" -Method POST -ContentType "application/json" -Body $body -TimeoutSec 30; Write-Host "Modelo testado: $model" -ForegroundColor Cyan; if($res.message.content){ Write-Host $res.message.content -ForegroundColor Green } else { Write-Host "Ollama respondeu, mas sem conteudo de chat." -ForegroundColor Yellow } } catch { Write-Host "Falha no teste do Ollama/modelo. Talvez o modelo nao suporte chat ou esteja offline." -ForegroundColor Yellow; Write-Host $_.Exception.Message }
```

Abrir pasta do projeto

```
$project="C:\Users\Jeanc\MestreDoPC_V7"; Start-Process $project; Write-Host "Pasta do projeto aberta: $project" -ForegroundColor Green
```

Abrir pasta de logs

```
$project="C:\Users\Jeanc\MestreDoPC_V7"; $logDir=Join-Path $project "logs"; New-Item -ItemType Directory -Force -Path $logDir | Out-Null; Start-Process $logDir; Write-Host "Pasta de logs aberta: $logDir" -ForegroundColor Green
```

Abrir TEMP do usuário

```
Start-Process $env:TEMP; Write-Host "TEMP aberto: $env:TEMP" -ForegroundColor Green
```

Abrir Startup do usuário

```
$startup=[Environment]::GetFolderPath("Startup"); Start-Process $startup; Write-Host "Startup aberto: $startup" -ForegroundColor Green
```

Ver tarefas MestreDoPC

```
$names=@("MestreDoPC_Admin_Launcher","MestreDoPC_Startup"); $rows=foreach($name in $names){ $task=Get-ScheduledTask -TaskName $name -ErrorAction SilentlyContinue; if($task){ $info=Get-ScheduledTaskInfo -TaskName $name -ErrorAction SilentlyContinue; [pscustomobject]@{Task=$name;State=$task.State;LastRun=$info.LastRunTime;LastResult=$info.LastTaskResult;NextRun=$info.NextRunTime} } else { [pscustomobject]@{Task=$name;State="Nao encontrada";LastRun="";LastResult="";NextRun=""} } }; $rows | Format-Table -AutoSize
```

Checkup Geral (Resumo Completo)

```
$os=Get-WmiObject Win32_OperatingSystem; $cpu=Get-WmiObject Win32_Processor; $disk=Get-PSDrive C; $ram=[math]::Round($os.FreePhysicalMemory/1MB,1); $totalRam=[math]::Round($os.TotalVisibleMemorySize/1MB,1); $diskFree=[math]::Round($disk.Free/1GB,1); $diskTotal=[math]::Round(($disk.Used+$disk.Free)/1GB,1); $up=[Management.ManagementDateTimeConverter]::ToDateTime($os.LastBootUpTime); $days=([datetime]::Now-$up).Days; Write-Host "=== CHECKUP DO PC ===" -ForegroundColor Cyan; Write-Host "CPU: $($cpu.Name)" -ForegroundColor White; Write-Host "RAM: $ram GB livre de $totalRam GB" -ForegroundColor $(if($ram -lt 2){"Red"}else{"Green"}); Write-Host "Disco C: $diskFree GB livre de $diskTotal GB" -ForegroundColor $(if($diskFree -lt 20){"Red"}else{"Green"}); Write-Host "Uptime: $days dias" -ForegroundColor $(if($days -gt 7){"Yellow"}else{"Green"}); Write-Host "Windows: $($os.Caption) Build $($os.BuildNumber)" -ForegroundColor White
```

Verificar Saúde da Bateria

```
$desktop = [Environment]::GetFolderPath('DesktopDirectory'); $report = Join-Path $desktop 'battery-report.html'; powercfg /batteryreport /output "$report"; Write-Host "📄 Relatório da bateria salvo em: $report" -ForegroundColor Green
```

Relatório de Energia Completo

```
$desktop = [Environment]::GetFolderPath('DesktopDirectory'); $report = Join-Path $desktop 'energy-report.html'; powercfg /energy /output "$report"; Write-Host "📄 Relatório de energia salvo em: $report (aguarde 60s para completar)!" -ForegroundColor Green
```

Verificar Integridade do Sistema (SFC + DISM)

```
Write-Host "Etapa 1/2: DISM..." -ForegroundColor Yellow; DISM /Online /Cleanup-Image /ScanHealth; Write-Host "Etapa 2/2: SFC..." -ForegroundColor Yellow; sfc /scannow; Write-Host "✅ Verificação completa!" -ForegroundColor Green
```

Verificar Atualizações Pendentes

```
$updates = (New-Object -ComObject Microsoft.Update.Session).CreateUpdateSearcher().Search("IsInstalled=0"); if($updates.Updates.Count -gt 0) { Write-Host "$($updates.Updates.Count) atualizações pendentes:" -ForegroundColor Yellow; $updates.Updates | ForEach-Object { Write-Host " - $($_.Title)" } } else { Write-Host "✅ Sistema atualizado!" -ForegroundColor Green }
```

Verificar Versão da BIOS/UEFI

```
Get-WmiObject Win32_BIOS | Select-Object Manufacturer,Name,SMBIOSBIOSVersion,ReleaseDate | Format-List
```

Verificar Saúde do Disco (SMART)

```
Get-PhysicalDisk | Select-Object FriendlyName,MediaType,HealthStatus,OperationalStatus,@{N="Tamanho (GB)";E={ [math]::Round($_.Size/1GB,1) }} | Format-Table -AutoSize
```

Listar Drivers Desatualizados

```
Get-WmiObject Win32_PnPSignedDriver | Where-Object {$_.DriverDate} | Select-Object DeviceName,DriverVersion,@{N="Data";E={ [Management.ManagementDateTimeConverter]::ToDateTime($_.DriverDate).ToString("dd/MM/yyyy") }} | Sort-Object Data | Select-Object -First 15 | Format-Table -AutoSize
```

Verificar Erros Recentes do Sistema

```
Get-WinEvent -FilterHashtable @{LogName="System";Level=2;StartTime=(Get-Date).AddDays(-7)} -MaxEvents 15 -EA 0 | Select-Object TimeCreated,Id,Message | Format-Table -Wrap -AutoSize
```

Tempo de Boot do Windows

```
$boot = (Get-WinEvent -FilterHashtable @{LogName="System";Id=6005} -MaxEvents 1).TimeCreated; $ready = (Get-WinEvent -FilterHashtable @{LogName="System";Id=6013} -MaxEvents 1 -EA 0); Write-Host "Último boot: $boot" -ForegroundColor Cyan
```

8. Diagnóstico (9 comandos)

Informações do Sistema

```
Get-ComputerInfo | Select-Object WindowsProductName,WindowsVersion,OsArchitecture,CsName,CsProcessors,CsTotalPhysicalMemory | Format-List
```

Temperatura do CPU (WMI)

```
Get-WmiObject MSAcpi_ThermalZoneTemperature -Namespace "root/wmi" | ForEach-Object { Write-Host "Zona: $($_.InstanceName) -> $($(_.CurrentTemperature - 2732)/10)°C" }
```

Histórico de Erros do Sistema

```
Get-EventLog -LogName System -EntryType Error -Newest 20 | Select-Object TimeGenerated,Source,Message | Format-Table -AutoSize
```

Uptime do Sistema

```
$boot=(Get-WmiObject Win32_OperatingSystem).LastBootUpTime; $up=[Management.ManagementDateTimeConverter]::ToDateTime($boot); Write-Host "Ligado desde: $up ($([datetime]::Now - $up).Days)d $([datetime]::Now - $up).Hours)h" -ForegroundColor Cyan
```

Placa de Vídeo (GPU)

```
Get-WmiObject Win32_VideoController | Select-Object Name,DriverVersion,@{N="VRAM (GB)";E={ [math]::Round($_.AdapterRAM/1GB,2) }} | Format-Table -AutoSize
```

Inventário de Software Instalado

```
Get-WmiObject Win32_Product | Select-Object Name,Version | Sort-Object Name | Format-Table -AutoSize
```

Verificar Drivers Problemáticos

```
Get-WmiObject Win32_PnPEntity | Where-Object ConfigManagerErrorCode -ne 0 | Select-Object Name,ConfigManagerErrorCode | Format-Table -AutoSize
```

Teste de Memória RAM (Agendar)

```
MdSched.exe
```

Relatório de Energia do Sistema

```
$desktop = [Environment]::GetFolderPath('DesktopDirectory'); $report = Join-Path $desktop 'energy-report.html'; powercfg /energy /output "$report"; Write-Host "✅ Relatório salvo em: $report" -ForegroundColor Green
```

9. Desligamento (9 comandos)

Desligar Agora

```
Stop-Computer -Force
```

Reiniciar Agora

```
Restart-Computer -Force
```

Desligar em 60 segundos

```
shutdown /s /t 60; Write-Host "⏰ PC desligará em 60s. Use shutdown /a para cancelar." -ForegroundColor Yellow
```

Cancelar Desligamento Agendado

```
shutdown /a; Write-Host "✅ Desligamento cancelado!" -ForegroundColor Green
```

Suspender o PC

```
Add-Type -AssemblyName System.Windows.Forms; [System.Windows.Forms.Application]::SetSuspendState("Suspend",$false,$false)
```

Hibernar o PC

```
shutdown /h
```

Sair da Conta do Usuário

```
logoff
```

Desligar em 30 minutos

```
shutdown /s /t 1800; Write-Host "🔴 PC desligará em 30 minutos." -ForegroundColor Yellow
```

Reiniciar para BIOS/UEFI

```
shutdown /r /fw /t 5; Write-Host "🔴 Reiniciando para UEFI em 5s..." -ForegroundColor Cyan
```

10. Git / Mestre do PC (10 comandos)

Git Status do Projeto

```
Set-Location "C:\MestreDoPC_V7"; git status
```

Git Add + Commit + Push

```
Set-Location "C:\MestreDoPC_V7"; git add -A; git commit -m "feat: atualização automática $(Get-Date -Format dd/MM/yyyy)"; git push; Write-Host "✅ Push realizado!" -ForegroundColor Green
```

Git Pull (Atualizar Local)

```
Set-Location "C:\MestreDoPC_V7"; git pull; Write-Host "✅ Repositório atualizado!" -ForegroundColor Green
```

Git Log (Últimos 10)

```
Set-Location "C:\MestreDoPC_V7"; git log --oneline -10
```

Ver Branch Atual

```
Set-Location "C:\MestreDoPC_V7"; git branch -v
```

Configurar Git (nome/email)

```
git config --global user.name "Jeanc"; git config --global user.email "seu@email.com"; Write-Host "✅ Git configurado!" -ForegroundColor Green
```

Ver Configuração do Git

```
git config --list --global
```

Clonar Repositório

```
git clone https://github.com/USUARIO/REPO.git "C:\Projetos\REPO"; Write-Host "✅ Clonado!" -ForegroundColor Green
```

Git Stash (Salvar mudanças temporariamente)

```
Set-Location "C:\MestreDoPC_V7"; git stash; Write-Host "✅ Mudanças salvas no stash!" -ForegroundColor Green
```

Git Stash Pop (Restaurar mudanças)

```
Set-Location "C:\MestreDoPC_V7"; git stash pop; Write-Host "✅ Mudanças restauradas!" -ForegroundColor Green
```

11. Remoção Específica (8 comandos)

Remover Bloatware Xbox

```
Get-AppxPackage *xboxapp* | Remove-AppxPackage -EA 0; Get-AppxPackage *xboxidentityprovider* | Remove-AppxPackage -EA 0; Write-Host "✅ Xbox apps removidos!" -ForegroundColor Green
```

Remover Cortana

```
Get-AppxPackage -AllUsers Microsoft.549981C3F5F10 | Remove-AppxPackage -EA 0; Write-Host "✅ Cortana removida!" -ForegroundColor Green
```

Remover Aplicativos Padrão Desnecessários

```
"Microsoft.3DBuilder", "Microsoft.BingNews", "Microsoft.BingWeather", "Microsoft.GetHelp", "Microsoft.Getstarted", "Microsoft.MixedReality.Portal", "Microsoft.SkypeApp", "Microsoft.Todos", "Microsoft.WindowsFeedbackHub", "Microsoft.ZuneMusic", "Microsoft.ZuneVideo" | ForEach-Object { Get-AppxPackage $_ | Remove-AppxPackage -EA 0 }; Write-Host "✅ Bloatware removido!" -ForegroundColor Green
```

Desativar Anúncios na Tela de Bloqueio

```
Set-ItemProperty "HKCU:\Software\Microsoft\Windows\CurrentVersion\ContentDeliveryManager" -Name "RotatingLockScreenOverlaysEnabled" -Value 0; Write-Host "✅ Anúncios desativados!" -ForegroundColor Green
```

Desativar Sugestões no Menu Iniciar

```
Set-ItemProperty "HKCU:\SOFTWARE\Microsoft\Windows\CurrentVersion\ContentDeliveryManager" -Name "SystemPaneSuggestionsEnabled" -Value 0; Write-Host "✅ Sugestões desativadas!" -ForegroundColor Green
```

Desinstalar OneDrive

```
taskkill /f /im OneDrive.exe -EA 0; Start-Process "$env:SystemRoot\SysWOW64\OneDriveSetup.exe" -ArgumentList "/uninstall" -Wait -EA 0; Write-Host "✅ OneDrive desinstalado!" -ForegroundColor Green
```

Remover Telemetria da Microsoft

```
sc stop DiagTrack; sc config DiagTrack start= disabled; sc stop dmwappushservice; sc config dmwappushservice start= disabled; Write-Host "✅ Telemetria desativada!" -ForegroundColor Green
```

Listar Todos os Apps Instalados

```
Get-AppxPackage -AllUsers | Select-Object Name,PackageFullName | Sort-Object Name | Format-Table -AutoSize
```

12. Segurança Geral (9 comandos)

Verificar Status do Windows Defender

```
Get-MpComputerStatus | Select-Object AMRunningMode,AntivirusEnabled,RealTimeProtectionEnabled,AntivirusSignatureLastUpdated | Format-List
```

Atualizar Definições do Defender

```
Update-MpSignature; Write-Host "✅ Definições atualizadas!" -ForegroundColor Green
```

Scan Rápido com Defender

```
Start-MpScan -ScanType QuickScan; Write-Host "✅ Scan rápido iniciado!" -ForegroundColor Green
```

Scan Completo com Defender

```
Start-MpScan -ScanType FullScan; Write-Host "✅ Scan completo iniciado!" -ForegroundColor Green
```

Verificar Firewall do Windows

```
Get-NetFirewallProfile | Select-Object Name,Enabled | Format-Table
```

Ativar Firewall em todos os perfis

```
Set-NetFirewallProfile -Profile Domain,Public,Private -Enabled True; Write-Host "✅ Firewall ativado!" -ForegroundColor Green
```

Ver Regras de Firewall Ativas

```
Get-NetFirewallRule | Where-Object Enabled -eq True | Select-Object DisplayName,Direction,Action | Format-Table -AutoSize
```

Checar Portas Abertas

```
netstat -ano | Where-Object { $_ -match "LISTENING" } | Select-Object -First 20
```

Listar Processos Suspeitos (sem assinatura)

```
Get-Process | Where-Object { -not (Get-AuthenticodeSignature $_.Path -EA 0).IsOSBinary } | Select-Object Name,Path,Id | Format-Table -AutoSize
```

13. Rede Segura (8 comandos)

DNS Cloudflare Seguro (1.1.1.1)

```
Set-DnsClientServerAddress -InterfaceIndex ((Get-NetAdapter | Where-Object Status -eq "Up").InterfaceIndex) -ServerAddresses "1.1.1.1","1.0.0.1"; Write-Host "✅ DNS Cloudflare definido!" -ForegroundColor Green
```

DNS Google (8.8.8.8)

```
Set-DnsClientServerAddress -InterfaceIndex ((Get-NetAdapter | Where-Object Status -eq "Up").InterfaceIndex) -ServerAddresses "8.8.8.8","8.8.4.4"; Write-Host "✅ DNS Google definido!" -ForegroundColor Green
```

DNS OpenDNS (Família)

```
Set-DnsClientServerAddress -InterfaceIndex ((Get-NetAdapter | Where-Object Status -eq "Up").InterfaceIndex) -ServerAddresses "208.67.222.123","208.67.220.123"; Write-Host "✅ DNS OpenDNS Family definido!" -ForegroundColor Green
```

Restaurar DNS Automático (DHCP)

```
Set-DnsClientServerAddress -InterfaceIndex ((Get-NetAdapter | Where-Object Status -eq "Up").InterfaceIndex) -ResetServerAddresses; Write-Host "✅ DNS automático restaurado!" -ForegroundColor Green
```

Bloquear Telemetria Microsoft no Hosts

```
$hosts = "C:\Windows\System32\drivers\etc\hosts"; $entries = @("0.0.0.0 telemetry.microsoft.com","0.0.0.0 vortex.data.microsoft.com","0.0.0.0 settings-win.data.microsoft.com"); $entries | ForEach-Object { if (!(Select-String -Path $hosts -Pattern $_ -Quiet)) { Add-Content $hosts $_ } }; Write-Host "✅ Telemetria bloqueada no hosts!" -ForegroundColor Green
```

Ver DNS Atual

```
Get-DnsClientServerAddress | Where-Object AddressFamily -eq 2 | Format-Table InterfaceAlias,ServerAddresses -AutoSize
```

Desativar IPv6

```
Disable-NetAdapterBinding -Name "*" -ComponentID ms_tcpip6 -EA 0; Write-Host "✅ IPv6 desativado!" -ForegroundColor Green
```

Verificar VPN/Proxy ativo

```
netsh winhttp show proxy; Get-ItemProperty "HKCU:\Software\Microsoft\Windows\CurrentVersion\Internet Settings" | Select-Object ProxyEnable,ProxyServer
```


Mestre do PC V8 - Preview

Visao Geral

Interface moderna com cards 3D, dashboard de metricas, log de atividades, terminal e chat IA Ollama. Conecta ao launcher HTTP via endpoints /ping, /run, /run-status, /status, /ollama/tags, /ollama/chat.

Funcoes JavaScript

- nowTime(): retorna hora local no formato pt-BR.
- log(msg, type): adiciona entrada no terminal de logs.
- renderCards(): renderiza cards de Otimizacao e Ferramentas com efeito 3D.
- runCommand(cmd, label, card): envia comando para /run e inicia polling.
- pollJob(jobId, label, card): consulta /run-status ate conclusao.
- checkPing(): verifica saude do launcher via /ping.
- loadMetrics(): busca /status e renderiza CPU, RAM, disco e temperatura.
- openTerminal(): abre terminal enviando comando inicial.
- checkOllama(): lista modelos disponiveis e preenche select.
- escapeHtml(s): escapa caracteres HTML.
- renderMarkdown(text): converte markdown basico e blocos de codigo.
- addMsg(role, text): adiciona mensagem ao chat IA.
- sendChat(): envia mensagem para /ollama/chat com streaming.
- init(): inicializa app, listeners e intervalos.
- make(): helper interno para criar cards (arrow function).

Comandos PowerShell

Otimizacao: Limpeza Rapida

```
Remove-Item "$env:TEMP\*" -Recurse -Force -EA 0; Remove-Item "C:\Windows\Temp\*" -Recurse -Force -EA 0; Clear-RecycleBin -Force -EA 0; Write-Host "Limpeza concluida!" -ForegroundColor Green
```

Otimizacao: Liberar RAM

```
[System.GC]::Collect(); [System.GC]::WaitForPendingFinalizers(); Write-Host "RAM liberada!" -ForegroundColor Green
```

Otimizacao: Reparar Sistema

```
sfc /scannow; DISM /Online /Cleanup-Image /RestoreHealth
```

Ferramentas: Windows Defender

```
Get-MpComputerStatus | Select-Object AMRunningMode,AntivirusEnabled,RealTimeProtectionEnabled | Format-List; Start-MpScan -ScanType QuickScan
```

Ferramentas: Diagnostico de Rede

```
ipconfig /flushdns; ipconfig /release; ipconfig /renew; Test-Connection 8.8.8.8 -Count 2 | Format-Table -AutoSize
```

Ferramentas: Informacoes do Sistema

```
Get-ComputerInfo | Select-Object WindowsProductName,WindowsVersion,OsArchitecture,CsName,CsProcessors | Format-List
```

Ferramentas: Espaco em Disco

```
Get-PSDrive -PSProvider FileSystem | Select-Object Name,@{N="Usado (GB)";E={ [math]::Round((($_.Used/1GB),2)}},@{N="Livre (GB)";E={ [math]::Round((($_.Free/1GB),2)}},@{N="Total (GB)";E={ [math]::Round(((($_.Used+$_.Free)/1GB),2)} } | Format-Table -AutoSize
```

Constantes Importantes

- API endpoints: /ping, /run, /run-status, /status, /ollama/tags, /ollama/chat
- SYSTEM_PROMPT: define personalidade do Mestre do PC em portugues brasileiro.

Mestre do PC V9 - Designs

Visao Geral

Interface futurista/cyberpunk com modulos core, dashboard de status, terminal modal e chat IA com botao EXECUTAR em blocos de codigo. Usa endpoints /ping, /run, /run-status, /status, /ollama/tags, /ollama/chat.

Funcoes JavaScript

- nowStr(): retorna hora como HH:MM:SS.
- addLog(text, type): insere entrada no log de atividades.
- api(path, opts): wrapper de fetch generico.
- getJson(path): wrapper fetch JSON.
- pingSystem(): atualiza badge SYSTEM ONLINE/OFFLINE.
- fmtUptime(sec): formata segundos em dias/horas/minutos.
- loadStatus(): carrega CPU, RAM livre, disco livre e uptime.
- renderModules(): renderiza 6 cards de modulos core.
- runModule(m, card): envia comando do modulo e exhibe no terminal.
- pollJob(jobId, title): polling de status com output ao vivo.
- sleep(ms): utilidade de espera.

- `openTerminal(title)`: abre modal de terminal.
- `setTerminal(text)`: define conteudo do terminal.
- `setChatStatus(on)`: atualiza indicador online/offline.
- `addMsg(role, text)`: adiciona mensagem ao chat.
- `renderMarkdown(t)`: converte markdown incluindo blocos com botao EXECUTAR.
- `esc(s)`: escapa HTML interno do `renderMarkdown`.
- `checkOllama()`: lista modelos Ollama.
- `openChat()`: abre chat com mensagem de boas-vindas.
- `closeChat()`: fecha chat.
- `sendChat()`: envia pergunta com historico limitado aos ultimos 12 tumos.

Comandos PowerShell (Core Modules)

System Clean - Purge temp files and clear system cache instantly.

```
Remove-Item "$env:TEMP\*" -Recurse -Force -EA 0; Remove-Item "C:\Windows\Temp\*" -Recurse -Force -EA 0; Clear-RecycleBin -Force -EA 0; Write-Host "OK Limpeza concluida" -ForegroundColor Green
```

RAM Boost - Free memory with .NET Garbage Collector injection.

```
[System.GC]::Collect(); [System.GC]::WaitForPendingFinalizers(); Write-Host "OK RAM liberada" -ForegroundColor Green
```

System Repair - Run SFC and DISM to fix corrupted Windows files.

```
sfc /scannow; Write-Host ""; DISM /Online /Cleanup-Image /RestoreHealth; Write-Host "OK Reparo concluido" -ForegroundColor Green
```

Network Diagnostic - ipconfig /all, flush DNS and renew IP.

```
ipconfig /all; Write-Host ""; ipconfig /flushdns; ipconfig /release; ipconfig /renew; Write-Host "OK Rede renovada" -ForegroundColor Green
```

Defender Scan - Quick scan with Windows Defender.

```
Start-MpScan -ScanType QuickScan; Write-Host "OK Scan rapido iniciado" -ForegroundColor Green
```

System Info - Show Windows product, CPU, RAM and disks.

```
Get-ComputerInfo | Select-Object WindowsProductName,WindowsVersion,OsArchitecture,CsName,CsProcessors,CsTotalPhysicalMemory | Format-List; Get-PSDrive -PSProvider FileSystem | Select-Object Name,@{N="Usado (GB)";E={ [math]::Round(($_.Used/1GB),2) }},@{N="Livre (GB)";E={ [math]::Round(($_.Free/1GB),2) }} | Format-Table -AutoSize
```

Diferenciais do V9

- Botao EXECUTAR embutido em blocos de codigo gerados pela IA.
- Historico de chat truncado para ultimos 12 tumos.
- Terminal modal dedicado para cada modulo.
- Design cyberpunk com neon cyan/magenta.